significant bloat problems. The database had tables with very high amounts of Updates causing dead tuples to accumulate quickly. These changes helped Autovacuum run more effectively.

Another example from Sentry.io in the post *Transaction ID Wraparound in Postgres*,[6] they describe lowering `autovacuum_vacuum_cost_delay` down to `0ms` because Vacuum was not running frequently enough.

You've now seen how to adjust Autovacuum. Shift your focus to the Indexes on your tables to make sure they're maintained well.

Rebuilding Indexes Without Downtime

Periodically rebuilding an index in PostgreSQL ensures it only has pointers to live tuples. As a refresher, Indexes are secondary data stores that are owned by tables.

To rebuild Indexes in PostgreSQL you use the `REINDEX` statement. Dropping and creating the index again achieves the same outcome as it builds the index based on live tuples. But on a live system, this is unsafe because it means the index won't be available for active queries that use it.

On older versions of PostgreSQL, rebuilding indexes while the server was running was not possible. An index could be safely rebuilt (Reindexed) if the database was taken down without disrupting any queries.

Fortunately PostgreSQL has made it possible to rebuild indexes while the server is running. The index remains available to live queries that are using it.

If you're running PostgreSQL version 12 or older you won't yet have access to this capability. Upgrading to PostgreSQL 13 or newer is the best option because there are many improvements besides online index rebuilds. If upgrading is not an option, you've got alternatives. Use the extension and command line program `pg_repack`[7] to perform online index rebuilds.

The post *Using 'pg_repack' to Rebuild Indexes*[8] provides an overview of how to use pg_repack to replace Indexes online. Briefly, pg_repack can be used to rebuild an index. The technique it uses is to rebuild a replacement index and then swap in the replacement while dropping the old index in a transaction. pg_repack works well for that purpose but it is a separate client program from

6. https://blog.sentry.io/2015/07/23/transaction-id-wraparound-in-postgres/
7. https://reorg.github.io/pg_repack/
8. https://andyatkinson.com/blog/2021/09/28/pg-repack